

RadiologyAI

AI Disclosure Appendix

Fair Use of AI — Code of Conduct Compliance

V G Masilamani(DA25S005)

AI Tool Used: Claude (Anthropic) — claude.ai

Purpose: Conceptual clarification, syntax help, bug debugging,
UI boilerplate, and non-MLOps configuration assistance

Core MLOps Logic: Designed, implemented, and understood by the student

Compliance: Adheres to DA5402 Code of Conduct: Fair Use of AI

Contents

1	Overview of AI Usage	1
2	Detailed AI Usage Log	1
2.1	Category 1 — Conceptual Understanding (Allowed)	1
2.2	Category 2 — Syntax Clarification (Allowed)	2
2.3	Category 3 — Bug Debugging (Allowed)	2
2.4	Category 4 — Non-MLOps Boilerplate (Allowed)	2
3	What Was NOT Done Using AI	3
4	Compliance Summary	3

Overview of AI Usage

In accordance with the DA5402 Code of Conduct on Fair Use of AI, this appendix discloses all instances where Claude (Anthropic) was consulted during the development of the RadiologyAI project. AI assistance was limited to:

- **Conceptual understanding** — asking what concepts mean
- **Syntax clarification** — asking for correct API/framework syntax
- **Bug debugging** — explaining error messages
- **Non-MLOps boilerplate** — React components, nginx config, Docker
- **Documentation structuring** — proofreading and formatting help

AI was **NOT** used for: pipeline design, DVC stages, MLflow schema, Airflow DAG logic, Prometheus rules, model training logic, or metric generation.

Detailed AI Usage Log

Category 1 — Conceptual Understanding (Allowed)

Question	How I Used It	Status
Explain data drift detection	Understood the concept, then chose pixel statistics (mean, std, brightness) and wrote <code>drift_detection.py</code> myself	Allowed
What is Grad-CAM?	Understood the theory, then integrated <code>pytorch-grad-cam</code> into <code>EfficientNetB0</code> myself	Allowed
DVC stages vs pipelines	Understood <code>dvc.yaml</code> structure, then designed 9-stage pipeline based on project needs	Allowed
MLflow autolog vs manual logging	Understood autolog behavior, then decided metrics to log (VRAM, dataset hash, git commit)	Allowed
CosineAnnealingLR in PyTorch	Understood scheduler behavior, then chose it over StepLR for fine-tuning	Allowed
Weighted cross-entropy loss	Understood class imbalance handling, then computed weights: Normal=2.197, Pneumonia=0.760, COVID=0.814	Allowed
Prometheus metric types	Understood Counter/Gauge/Histogram, then chose appropriate types for <code>feedback_total</code> , accuracy, latency	Allowed
Airflow XCom system	Understood task communication, then designed feedback accuracy check with XCom branching	Allowed
TorchScript for serving	Understood export format, then wrote <code>export.py</code> to trace and save model	Allowed

Question	How I Used It	Status
MLflow Model Registry	Understood staging (Production/Staging/Archived), then wrote <code>register_model.py</code> for promotion by <code>val_f1</code>	Allowed

Category 2 — Syntax Clarification (Allowed)

Question	How I Used It	Status
FastAPI file upload syntax	Understood <code>UploadFile</code> and <code>File()</code> , then wrote <code>/predict</code> endpoint with custom validation	Allowed
Pydantic BaseSettings	Understood Settings pattern, then designed Settings class with needed fields	Allowed
FastAPI lifespan context manager	Understood <code>asyncontextmanager</code> , then wrote lifespan to load model and restore gauge	Allowed
Docker volume mounting	Understood volume syntax, then wrote <code>docker-compose.yml</code> with correct mounts	Allowed
React hooks (<code>useState</code> , <code>useEffect</code>)	Understood hook patterns, then wrote all React components myself	Allowed
Prometheus Gauge creation	Understood <code>prometheus.client</code> , then decided metrics to measure and update timing	Allowed

Category 3 — Bug Debugging (Allowed)

Error	What I Did After	Status
<code>ModuleNotFoundError: No module 'ml'</code>	AI explained <code>PYTHONPATH</code> ; I set <code>export</code> myself and understood why	Allowed
<code>alembic: Can't locate revision</code>	AI explained MLflow version mismatch; I upgraded MLflow and fixed mounts	Allowed
<code>sqlite3: unable to open database file</code>	AI explained Docker permissions; I switched to named volumes	Allowed
<code>Airflow: No such file or directory</code>	AI explained volume mounting; I fixed <code>docker-compose</code> volumes	Allowed
<code>npm: package-lock.json out of sync</code>	AI explained <code>npm ci</code> vs <code>install</code> ; I updated Dockerfile	Allowed
<code>DVC: output does not exist</code>	AI explained <code>dvc remove</code> ; I re-downloaded via Kaggle CLI	Allowed
<code>Grafana: no disk data on macOS</code>	AI explained <code>node_exporter</code> differences; I wrote corrected Prometheus query	Allowed
Gauge reset on Docker restart	AI explained persistence; I implemented <code>load_feedback_from_file()</code> in lifespan	Allowed

Category 4 — Non-MLOps Boilerplate (Allowed)

Area	AI Help + My Work	Status
React UI components	AI: basic structure; Me: all styling, theme, layout, page logic	Allowed
nginx.conf	AI: basic config; Me: proxy_pass, port settings, customization	Allowed
ReportLab PDF generation	AI: SimpleDocTemplate API; Me: full layout, content, Grad-CAM embedding	Allowed
Alertmanager YAML	AI: route/receiver structure; Me: all alert rules and logic	Allowed
LaTeX formatting	AI: table syntax; Me: all content, structure, technical details	Allowed

What Was NOT Done Using AI

Component	My Own Work
DVC pipeline design	Decided 9 stages, dependencies, and dvc.yaml structure
MLflow tracking schema	Chose metrics, params, and artifacts to log beyond autolog
Two-phase training	Decided frozen/unfrozen epochs and learning rates
Class weight computation	Computed weights from dataset distribution myself
Airflow DAG branching	Designed feedback accuracy check and drift detection logic
Prometheus metrics	Chose which metrics to expose and at what granularity
Alert thresholds	Set 5% error rate, 200ms latency, 80% accuracy thresholds
Feedback loop design	Full architecture: feedback → gauge → alert → Airflow → retrain
Drift detection	Chose pixel statistics as drift features
All MLOps decisions	Every design choice was mine and explainable

Compliance Summary

Requirement	Status
Human-in-the-Loop: Core logic is my own	YES
AI used only for scaffolding & debugging	YES
AI Disclosure Appendix included	YES
Tool clearly documented	YES — Claude (Anthropic)
Prompts are specific and listed	YES — Section 2
AI usage areas identified	YES — Categories 1–4
Can explain any line of code	YES
No fabricated metrics	YES — All real
No AI-generated MLOps code	YES
Ready for viva	YES